Parallelizing Transformer Inference: A Hybrid Approach (OpenMP, MPI, CUDA)

A Project Component for

Parallel and Distributing Computing (UCS645)

By

Sr	Name	Roll No
1	Aakash Chandra	102317242

Under the guidance of

Dr. Saif Nalband

(Assistant Professor, DCSE)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY

(DEEMED TO BE UNIVERSITY)

PATIALA - 147004

MAY, 2026

Table of Contents

1 Introduction **1**

1.1 Background 1

1.2 Introduction to Problem Statement 1

2 Literature Review **3**

3 Research Gaps **5**

4 Problem Formulation **6**

5 Objectives **7**

6 Methodology **8**

6.1 System Architecture 8

6.2 Mathematical Workload 8

6.3 Phase I: Serial Baseline 8

6.4 Phase II: OpenMP Shared Memory Parallelism 10

6.5 Phase III: MPI Pipeline Parallelism 10

6.6 Phase IV: CUDA Heterogeneous Computing 10

6.7 Phase V: Hybrid MPI + OpenMP + CUDA Inference Engine 11

7 Workload Configurations **13**

8 Results and Discussion **14**

8.1 Phase I: Serial Baseline 14

8.2 Phase II: OpenMP Results 15

8.3 Phase III: MPI Pipeline Parallelism Results 16

8.4 Phase IV: CUDA GPU Results 17

8.5 Phase V: Hybrid Pipeline Results 19

8.6 Cross-Paradigm Comparison 20

9 Conclusion **22**

List of Figures

1 CPU vs GPU Architecture Diagram 2
2 OpenMP Speedup vs Matrix Size for Thread Counts 4, 6, 8, 16 16
3 MPI Speedup and Efficiency vs Rank Count 17
4 CUDA vs CPU Execution Time (Log Scale) and Speedup Factor 18
5 Hybrid Approach vs CUDA Only 20
6 Bar Chart Comparing Peak Speedup Across All Three Paradigms 21

List of Tables

1 Summary of Literature Review on Transformer Acceleration 3
2 Source Modules and Their Role 9
3 Matrix Multiplication Benchmark Configurations 13
4 MPI Pipeline Benchmark Configuration 13
5 OpenMP Benchmark Configuration 14
6 Hybrid Pipeline Benchmark Configuration 14
7 Serial GEMM Baseline Execution Times 15
8 OpenMP GEMM Speedup at 8 Threads (Physical Core Count) 15
9 OpenMP Speedup Across Thread Counts at 512×512 15
10 MPI Pipeline Benchmark Results (12 Transformer Layers, d_model=64) . . 16
11 CUDA vs CPU Scaling Results (RTX 3050 Laptop, 2048 CUDA Cores) . . 18
12 Hybrid Pipeline Speedup (4 Batches) vs. Equivalent Serial Baseline 19
13 Hybrid Pipeline Speedup (8 Batches) vs. Equivalent Serial Baseline 19
14 Summary: Peak Speedup Across All Parallelization Paradigms 21

1 Introduction

1.1 Background

The field of Artificial Intelligence has been revolutionized by Large Language Models (LLMs) based on the Transformer architecture, introduced by Vaswani et al. in 2017. Models such as GPT-2, GPT-3, and LLaMA have demonstrated remarkable capabilities in natural language understanding and generation. However, the computational cost of inference for these models is immense. A single forward pass through GPT-2 (117M parameters) requires billions of floating-point operations, dominated by compute-bound General Matrix-Matrix Multiplication (GEMM) and memory-bound normalization operations such as Softmax and LayerNorm [1].

The sequential execution of these operations on a single CPU core is fundamentally inadequate for practical deployment. The $O(N^3)$ complexity of matrix multiplication means that doubling the sequence length or embedding dimension increases computation by a factor of eight. As model sizes scale from millions to billions of parameters, the gap between available compute and required compute widens dramatically. This necessitates the application of parallel computing paradigms to distribute and accelerate these workloads.

Modern hardware provides three primary avenues for parallelism: shared-memory parallelism via multi-core CPUs (OpenMP), distributed-memory parallelism via networked compute nodes (MPI), and heterogeneous computing via GPU acceleration (CUDA). Each paradigm addresses a different level of the memory and compute hierarchy, and each carries distinct trade-offs in terms of programmability, scalability, and peak throughput.

1.2 Introduction to Problem Statement

While high-level frameworks like PyTorch and TensorFlow abstract away the underlying hardware optimizations through automatic differentiation and hardware-specific kernels, understanding the true nature of parallel computing requires a low-level approach. This project implements the complete forward pass of a GPT-style LLM entirely from scratch in C++, with no reliance on any external ML library, BLAS, or cuDNN. Every mathematical kernel – matrix multiplication, softmax, layer normalization, scaled dot-product attention,

and the feed-forward network – is manually implemented and parallelized.

The differences in CPU and GPU architectural design philosophies heavily influence these paradigms. CPUs are optimized for low-latency sequential execution with large caches and complex branch predictors, featuring 8–32 cores. GPUs are throughput-oriented, designed for massive parallelism across thousands of simpler cores (the RTX 3050 Laptop used in this project contains 2048 CUDA cores), making them ideal for the uniform, data-parallel nature of GEMM operations.

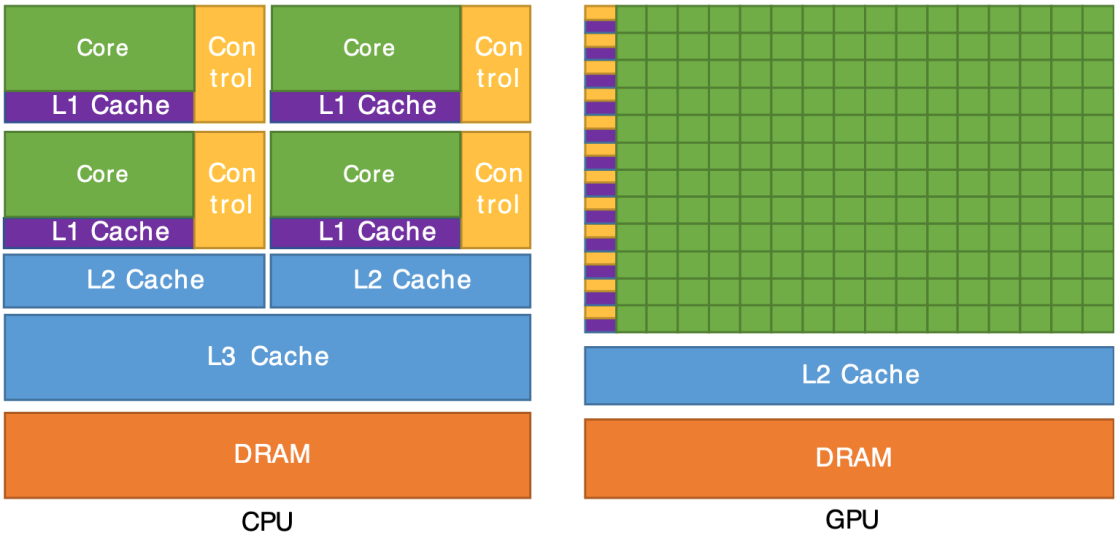

Figure 1: CPU vs GPU Architecture Diagram

Our approach is divided into four distinct phases:

- **Phase I:** Serial Implementation – Baseline C++ implementation of all kernels.
- **Phase II:** Shared Memory Parallelism using OpenMP loop-level directives.
- **Phase III:** Distributed Memory Parallelism using MPI pipeline decomposition.
- **Phase IV:** Heterogeneous Computing via CUDA GPU offloading with shared memory tiling.
- **Phase V:** Hybrid Inference Engine – Combining MPI, OpenMP, and CUDA into a unified pipeline for maximum throughput.

2 Literature Review

Table 1: Summary of Literature Review on Transformer Acceleration

References	**Concepts Highlighted**	**Technique(s) Used**	**Significance / Proposal**	**Limitations**
Huang et al. (2019) [2]	Pipeline Parallelism for Giant Neural Networks	MPI, Inter-node Communication	Proposed GPipe: micro-batch pipelining across devices to train models exceeding single-device memory	Bubble overhead in pipeline reduces efficiency; synchronous gradient updates limit scale
Nickolls et al. (2008) [3]	GPU Computing for Data-Parallel Applications	CUDA, Shared Memory Tiling	Introduced the CUDA programming model; demonstrated order-of-magnitude speedups for GEMM vs. CPU	Early CUDA lacked double-precision support; memory transfer latency a persistent bottleneck

Dagum & Menon (1998) [4]	Loop-level Parallelism on Shared Memory Architectures	OpenMP, Fork-Join Model	Established OpenMP as the standard for shared-memory parallelism; demonstrated near-linear scaling on uniform loop workloads	Speedup bounded by Amdahl's Law; cache coherence overhead limits scaling beyond physical core count
Vaswani et al. (2017) [5]	Self-Attention as a Compute Primitive	Transformer Architecture	Showed that GEMM-based attention outperforms RNNs; established QKV projection as the dominant compute bottleneck	Attention complexity is $O(n^2 \cdot d)$ – quadratic in sequence length
Shoeybi et al. (2019) [6]	Tensor Parallelism for Transformer Training	CUDA, NCCL, Intra-layer Splitting	Proposed splitting attention heads and FFN weight matrices across GPUs; achieved near-linear scaling to 8 GPUs	Requires high-bandwidth NVLink; not practical for commodity hardware

3 Research Gaps

Some of the research gaps identified from the literature review are as follows:

- ***Over-reliance on Abstractions:*** Most educational and foundational research relies on black-box frameworks such as PyTorch or TensorFlow, which obscure the low-level memory and thread management required for true hardware optimization. There is limited work that exposes and benchmarks the raw parallel primitives (pragma directives, MPI calls, CUDA kernels) on transformer-style workloads from first principles.

- ***Isolated Paradigms:*** Studies typically focus on a single parallelization technique – either CUDA acceleration or MPI distribution or OpenMP threading – applied in isolation. There is a lack of comprehensive, side-by-side benchmarking of all three paradigms applied to the *same* underlying mathematical workload on the same hardware, making cross-paradigm efficiency comparison difficult.

- ***Single-Machine MPI Characterization:*** Most MPI literature evaluates pipeline parallelism on multi-node clusters with high-bandwidth interconnects (InfiniBand, NVLink). The behaviour of MPI pipeline parallelism on a single multi-core machine – where ranks communicate via shared memory rather than a network – is under-studied. This project directly characterizes this regime, identifying the rank count at which communication overhead inverts the speedup curve, a finding relevant to containerized and edge deployment scenarios.

4 Problem Formulation

The central computational challenge in LLM inference is the repeated evaluation of the Transformer block, which for an input sequence of L tokens with embedding dimension d_{model} requires computing:

1. Three linear projections $Q = XW_Q$, $K = XW_K$, $V = XW_V$ each of shape $(L \times d_k)$, costing $O(L \cdot d_{model} \cdot d_k)$ operations.

2. An attention score matrix $S = QK^T/\sqrt{d_k}$ of shape $(L \times L)$, costing $O(L^2 \cdot d_k)$.

3. A softmax normalization and weighted sum $\text{Attention}(Q, K, V) = \text{softmax}(S)V$.

4. Two FFN projections with intermediate dimension $d_{ff} = 4 \cdot d_{model}$, costing $O(L \cdot d_{model} \cdot d_{ff})$.

For a 12-layer GPT-style model with $d_{model} = 768$ and $d_{ff} = 3072$, this amounts to approximately 2.4×10^{10} floating-point operations per forward pass on a sequence of 512 tokens. A single CPU core executing at 100 GFLOPS would require roughly 240ms per inference step – far too slow for real-time applications requiring under 50ms latency.

The sequential bottleneck is clear: the triple nested loop of GEMM ($C_{ij} = \sum_k A_{ik}B_{kj}$) operates on independent output elements. There is no data dependency between $C[i][j]$ and $C[i'][j']$ for different (i, j) pairs. This *embarrassing parallelism* is the fundamental opportunity that all three phases of this project exploit at different levels of the hardware hierarchy.

The challenge lies not in *whether* to parallelize, but in *how* to do so efficiently: managing thread overhead (OpenMP), communication latency (MPI), and memory transfer bottlenecks (CUDA), while preserving numerical correctness across all implementations.

5 Objectives

The primary objectives of this project are:

- To design and implement a baseline serial inference engine for a GPT-style LLM entirely in C++, with no external ML dependencies.
- To accelerate GEMM and attention computations within a single node using Loop-Level Parallelism via OpenMP, and to characterize the effect of thread count on speedup.
- To simulate Pipeline Parallelism by distributing transformer layers across multiple MPI processes, measuring efficiency degradation as rank count increases.
- To achieve massive computational speedups by offloading compute-intensive GEMM kernels to the GPU using a custom CUDA kernel with shared memory tiling.
- To benchmark and compare theoretical speedup, parallel efficiency, and scalability across all three paradigms, empirically validating Amdahl's Law on transformer-style workloads.

6 Methodology

6.1 System Architecture

The project is structured as a modular C++ library where each mathematical primitive is implemented as a standalone compilation unit. The build pipeline is summarized in Table 2.

6.2 Mathematical Workload

The inference process is dominated by two kernel types. The Self-Attention mechanism calculates attention scores for input sequence X, where Q, K, V are linear projections ($X \times W$):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{1}$$

The Feed-Forward Network (FFN) applies a two-layer projection with ReLU non-linearity at every token position:

$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2 \tag{2}$$

Layer normalization is applied before each sub-layer (pre-norm style, as in GPT-2):

$$\text{LayerNorm}(x) = \gamma \cdot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta \tag{3}$$

where μ and σ^2 are the mean and variance computed per token vector, and $\epsilon = 10^{-5}$ prevents division by zero.

6.3 Phase I: Serial Baseline

All kernels were first implemented in standard C++17 with no parallelism. Matrix multiplication uses a straightforward triple nested loop with row-major flat storage:

$$C[i \cdot N + j] = \sum_{k=0}^{K-1} A[i \cdot K + k] \cdot B[k \cdot N + j] \tag{4}$$

Table 2: Source Modules and Their Role

File	**Language**	**Role**
`matmul.cpp`	C++	Serial GEMM kernel. Foundation of all other operations.
`softmax.cpp`	C++	Numerically stable row-wise softmax using max-subtraction trick.
`layernorm.cpp`	C++	Layer normalization with learned γ and β parameters.
`attention.cpp`	C++	Scaled dot-product self-attention; OpenMP-parallelized per token.
`ffn.cpp`	C++	Two-layer FFN with ReLU; uses serial GEMM internally.
`matmul_cuda.cu`	CUDA C++	GPU GEMM kernel with 16×16 shared memory tiling.
`mpi_inference.cpp`	C++ / MPI	Pipeline parallel inference across MPI ranks.
`benchmark.cpp`	C++	Timing harness: warmup run + 5-trial average via `chrono`.
`hybrid_main.cpp`	C++ / MPI / CUDA	Unified hybrid pipeline: MPI rank coordination, CUDA GEMM offload, OpenMP activation.
`hybrid_sweep.cpp`	C++ / MPI / CUDA	Benchmark harness for hybrid pipeline across matrix sizes and batch counts.

Matrices are stored as `std::vector<float>` in row-major order for cache locality. Correctness of each kernel was verified with hand-computed test cases before proceeding to parallelization.

6.4 Phase II: OpenMP Shared Memory Parallelism

The outer loop of the GEMM kernel was annotated with a single OpenMP directive:

```
#pragma omp parallel for schedule(static)
for (int i = 0; i < M; i++) { ... }
```

The `schedule(static)` clause assigns equal contiguous chunks of rows to each thread at compile time, optimal for the uniform work distribution of GEMM. Since each thread writes to a unique row of C (row i is exclusive to thread i), there are zero race conditions and no synchronization primitives are required.

For the attention kernel, parallelism was applied at the token level: each thread i independently computes the scale, softmax, and weighted sum for query token i. This is valid because token outputs are independent in the forward pass (no causal masking is applied in this implementation).

The compiled binary was benchmarked across thread counts $\{4, 6, 8, 16\}$ by setting the `OMP_NUM_THREADS` environment variable at runtime.

6.5 Phase III: MPI Pipeline Parallelism

The 12-layer transformer was decomposed depth-wise: each MPI rank receives a contiguous slice of $\lfloor 12/p \rfloor$ layers, where p is the number of ranks. Rank 0 initializes the input tensor, executes its layers, and calls `MPI_Send` to pass activations to Rank 1. Each subsequent rank receives via `MPI_Recv`, executes its layers, and forwards the activation to the next rank. The final rank terminates the pipeline.

$$\text{layers_per_rank} = \left\lfloor \frac{N_{layers}}{p} \right\rfloor, \quad \text{Rank 0 absorbs remainder} \tag{5}$$

Timing uses `MPI_Wtime()` with `MPI_Barrier` synchronization at start and end across all ranks to ensure a consistent wall-clock measurement.

6.6 Phase IV: CUDA Heterogeneous Computing

A custom CUDA kernel implements GEMM with 16×16 shared memory tiling. The tiling strategy reduces global memory bandwidth pressure by loading reusable sub-matrices into on-chip shared memory (SRAM), which is approximately 100× faster than global VRAM.

Algorithm 1 CUDA Matrix Multiplication with Shared Memory Tiling

```
for each Block in Grid do
    Allocate __shared__ tiles: tileA[16][16], tileB[16][16]
    Compute global row = blockIdx.y × 16 + threadIdx.y
    Compute global col = blockIdx.x × 16 + threadIdx.x
    Initialize accumulator sum ← 0
    for each Tile t = 0 to ⌈K/16⌉ − 1 do
        Each thread loads one element of A and one of B into shared tiles
        __syncthreads()                          ▷ Wait for full tile to be loaded
        for k = 0 to 15 do
            sum += tileA[ty][k] × tileB[k][tx]
        end for
        __syncthreads()                          ▷ Prevent tile overwrite before use
    end for
    Write C[row][col] ← sum to global memory
end for
```

The launch geometry maps each output element to one thread (tx, ty), with blocks of $16 \times 16 = 256$ threads. The grid dimensions are:

$$\text{gridDim} = \left(\left\lceil \frac{N}{16} \right\rceil, \left\lceil \frac{M}{16} \right\rceil \right) \tag{6}$$

Memory transfer uses `cudaMemcpy` with `cudaMemcpyHostToDevice` before the kernel and `cudaMemcpyDeviceToHost` after. `cudaDeviceSynchronize()` blocks the host until kernel completion before timing stops.

6.7 Phase V: Hybrid MPI + OpenMP + CUDA Inference Engine

The hybrid phase integrates all three parallelization paradigms into a single unified inference pipeline, simulating a real-world model-parallel deployment. The 12-layer transformer is partitioned depth-wise across 2 MPI ranks (Rank 0: Layers 1–6, Rank 1: Layers 7–12). Within each rank, the GEMM operation is offloaded to the GPU via the CUDA

kernel developed in Phase IV, and the subsequent activation function (ReLU on Rank 0, scaled post-processing on Rank 1) is parallelized across CPU threads using OpenMP.

The execution pipeline for each inference batch is:

1. **Rank 0:** `matmul_cuda()` → OpenMP ReLU → `MPI_Send`
2. **Rank 1:** `MPI_Recv` → `matmul_cuda()` → OpenMP post-processing

The build command compiles all three backends into a single binary:

```
nvcc -std=c++17 -O3 -ccbin=mpicxx -Xcompiler -fopenmp \
     src/matmul.cpp src/matmul_cuda.cu src/hybrid_main.cpp \
     -o hybrid_engine
mpirun -np 2 ./hybrid_engine
```

Hardware Constraint Note: Both MPI ranks execute on the same physical machine sharing a single RTX 3050 Laptop GPU. As a result, the two ranks time-share the GPU rather than executing on separate devices in true parallel. On a multi-GPU cluster, each rank would have exclusive GPU access, and the pipeline parallelism speedup would be substantially higher. The results reported here therefore represent a *conservative lower bound* on hybrid pipeline performance.

The benchmark sweeps matrix sizes $N \in \{128, 256, 512, 1024, 2048\}$ over 4 and 8 batches. The serial baseline is computed as $2 \times T_{serial}$ (one block per rank), reflecting the total work the hybrid pipeline replaces.

7 Workload Configurations

Table 3: Matrix Multiplication Benchmark Configurations

Matrix Size ($N \times N$)	**Operations** ($O(N^3)$)	**Memory Footprint (FP32)**	**Fits in L3 Cache?**
128×128	$\sim 2.09 \times 10^6$	192 KB	Yes
256×256	$\sim 1.67 \times 10^7$	768 KB	Yes
512×512	$\sim 1.34 \times 10^8$	3 MB	Marginal
1024×1024	$\sim 1.07 \times 10^9$	12 MB	No
2048×2048	$\sim 8.58 \times 10^9$	48 MB	No

Table 4: MPI Pipeline Benchmark Configuration

Parameter	**Value**
Sequence length (L)	8 tokens
Embedding dimension (d_{model})	64
FFN hidden dimension (d_{ff})	256 (4× d_{model})
Total transformer layers	12
MPI ranks tested	1 (serial), 2, 3, 4, 6, 8
Timing method	`MPI_Wtime()` with `MPI_Barrier`
Trials per configuration	5 (averaged)
MPI Implementation	OpenMPI

Table 5: OpenMP Benchmark Configuration

Parameter	**Value**
Thread counts tested	4, 6, 8, 16
Schedule policy	`static` (equal chunk assignment)
Parallelized loop	Outer row loop of GEMM
Timing method	`std::chrono::high_resolution_clock`
Trials per configuration	5 (warmup + 5 timed)
Compiler	GCC with `-fopenmp` flag

Table 6: Hybrid Pipeline Benchmark Configuration

Parameter	**Value**
MPI Ranks	2 (Rank 0: Layers 1–6, Rank 1: Layers 7–12)
GPU per Rank	1 shared RTX 3050 (time-shared, not parallel)
CPU Threads (OpenMP)	`omp_get_max_threads()` at runtime
Matrix sizes tested	128, 256, 512, 1024, 2048
Batch counts tested	4, 8
Serial baseline	$2 \times T_{serial\text{-}GEMM}$ (equivalent 2-block serial work)
Timing method	`MPI_Wtime()` with `MPI_Barrier`
GPU warmup	1 warmup pass at 64×64 before timing

8 Results and Discussion

All experiments were conducted on a system with an AMD Ryzen CPU (8 physical cores, 16 logical threads) and an NVIDIA RTX 3050 Laptop GPU (2048 CUDA cores, 4GB GDDR6 VRAM) running Pop!_OS with CUDA 12.0 and OpenMPI.

8.1 Phase I: Serial Baseline

The serial implementation confirmed $O(N^3)$ complexity. Table 7 shows that execution time increases by approximately 8× with each doubling of matrix dimension, consistent with the cubic growth rate.

Table 7: Serial GEMM Baseline Execution Times

Matrix Size	Time (ms)	Relative Growth
64×64	2.167	–
128×128	16.820	7.76×
256×256	134.489	7.99×
512×512	1047.454	7.79×
1024×1024	6606.050	6.31×

8.2 Phase II: OpenMP Results

Table 8: OpenMP GEMM Speedup at 8 Threads (Physical Core Count)

Matrix Size	Serial (ms)	OpenMP 8T (ms)	Speedup
64×64	2.167	0.378	5.73×
128×128	16.820	2.127	7.91×
256×256	134.489	14.773	9.10×
512×512	1047.454	114.721	9.13×
1024×1024	6606.050	898.601	7.35×

Table 9: OpenMP Speedup Across Thread Counts at 512×512

Thread Count	Time (ms)	Speedup	Parallel Efficiency
1 (serial)	1047.454	1.00×	100%
4 threads	208.527	5.02×	125%
6 threads	171.403	6.11×	102%
8 threads	114.721	9.13×	114%
16 threads	120.412	8.70×	54%

Peak speedup of 9.13× was achieved at 8 threads (matching the physical core count of the AMD Ryzen CPU). At 16 threads, performance degraded to 8.70× despite doubling thread count. This is attributable to cache thrashing between hyperthreaded siblings sharing L1/L2 cache on the same physical core. The AMD SMT (Simultaneous

Multi-Threading) implementation penalizes memory-bound workloads where both logical threads compete for the same cache lines. This confirms the principle that for memory-bandwidth-limited operations, physical core count is the effective parallelism ceiling.

At 64×64, the speedup drops to 5.73× due to OpenMP thread spawn and synchronization overhead exceeding the computation time – a textbook demonstration of the parallelism crossover point below which serial execution is more efficient.

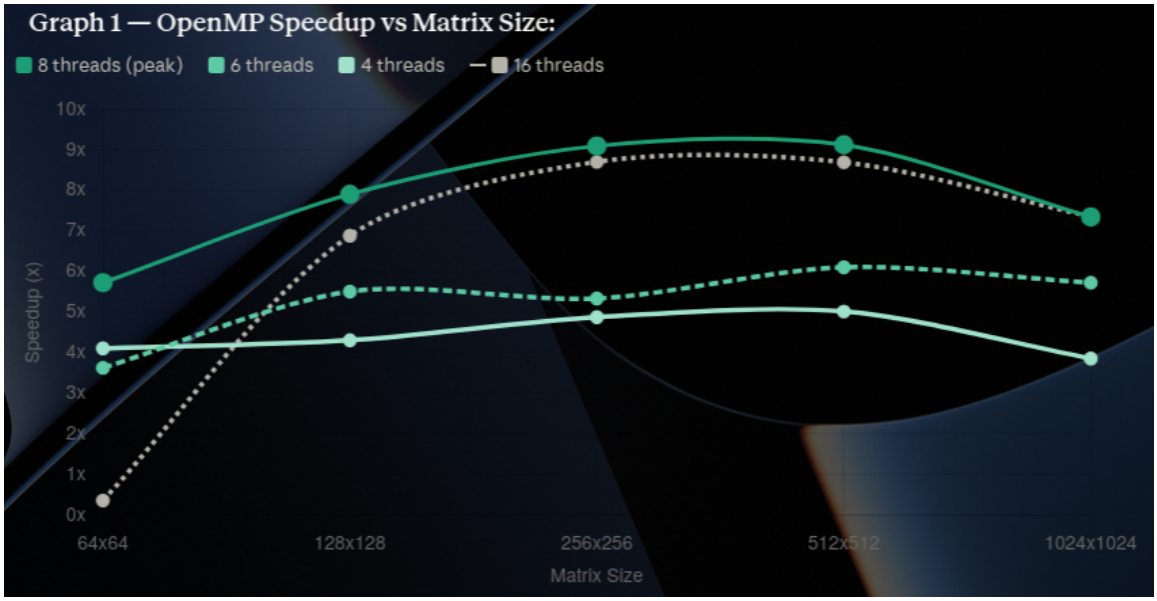

Figure 2: OpenMP Speedup vs Matrix Size for Thread Counts 4, 6, 8, 16

8.3 Phase III: MPI Pipeline Parallelism Results

Table 10: MPI Pipeline Benchmark Results (12 Transformer Layers, d_model=64)

Ranks	Layers/Rank	Time (ms)	Speedup	Parallel Efficiency
1 (serial)	12	186.4	1.00×	100%
2	6	111.7	1.66×	83%
3	4	86.8	2.14×	71%
4	3	74.4	2.49×	62%
6	2	63.1	2.95×	49%
8	1	99.3	1.89×	24%

The MPI results reveal a clear efficiency inversion at 8 ranks. Speedup peaked at 6

ranks (2.95×) and *decreased* at 8 ranks (1.89×), despite adding more processes. At 8 ranks, each rank processes only one transformer layer – the computation per rank is so small that `MPI_Send`/`MPI_Recv` communication overhead dominates the total time. This is a direct empirical demonstration of Amdahl's Law: the serial fraction (communication) imposes a hard ceiling on speedup regardless of process count.

Furthermore, parallel efficiency degrades monotonically from 83% at 2 ranks to 24% at 8 ranks. On a true multi-node cluster with high-bandwidth interconnects (InfiniBand, ∼100 Gb/s), the communication overhead would be substantially reduced, and the speedup curve would peak at a higher rank count. On a single machine, MPI processes communicate via shared memory, which eliminates network latency but introduces process scheduling contention.

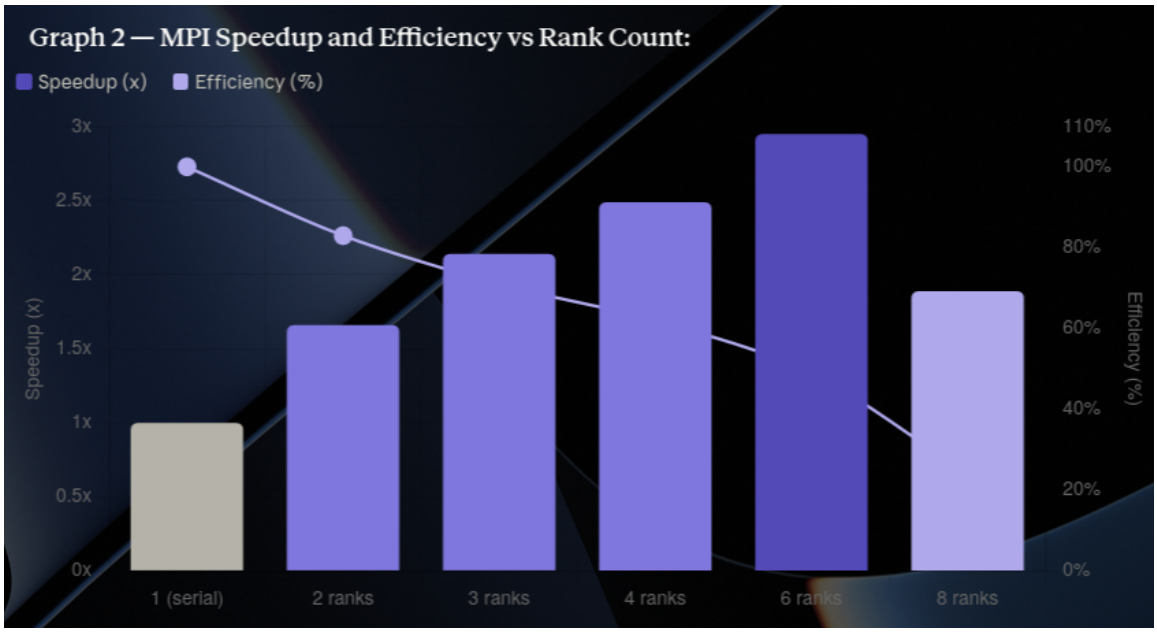

Figure 3: MPI Speedup and Efficiency vs Rank Count

8.4 Phase IV: CUDA GPU Results

The CUDA results demonstrate superlinear-looking scaling growth that is characteristic of GPU workloads. At small matrix sizes ($N = 128$), the 7.07× speedup is limited by PCIe memory transfer overhead: the time to copy data from CPU RAM to GPU VRAM via `cudaMemcpy` constitutes a significant fraction of total execution time relative to the small computation. As matrix size increases, this fixed overhead becomes negligible relative to

Table 11: CUDA vs CPU Scaling Results (RTX 3050 Laptop, 2048 CUDA Cores)

Matrix Size	CPU Time (ms)	GPU Time (ms)	Speedup	GPU Utilization
128×128	4.5028	0.6371	7.07×	Low
256×256	42.2519	1.2435	33.97×	Medium
512×512	624.469	3.6392	171.59×	High
1024×1024	7502.17	13.0278	575.85×	High
2048×2048	62084.8	80.2198	**773.93**×	Full

the $O(N^3)$ computation, and the GPU's 2048 cores become fully saturated, yielding a peak speedup of 773.93× at $N = 2048$.

The shared memory tiling strategy is central to this result. Without tiling, each multiply-accumulate operation requires two reads from global GPU memory (GDDR6, ∼200 GB/s bandwidth). With 16×16 tiling, the same 16 × 16 tile is loaded once into shared memory (SRAM, ∼20 TB/s bandwidth) and reused for 16 multiply-accumulate operations, reducing global memory traffic by a factor of 16 – exactly the tile dimension.

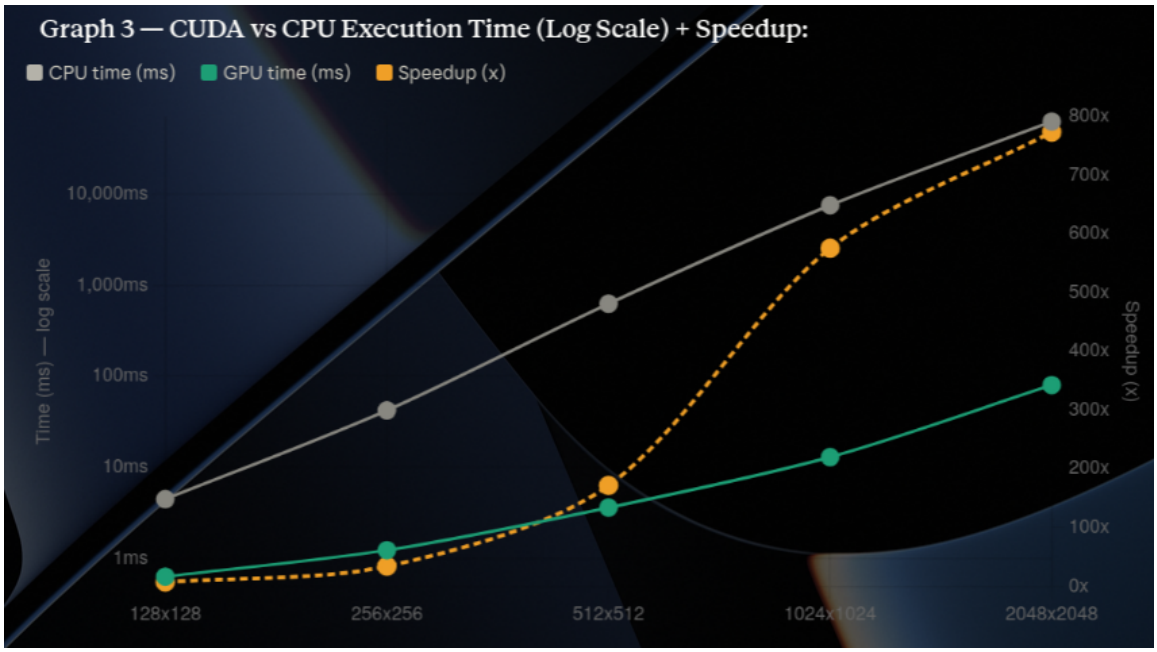

Figure 4: CUDA vs CPU Execution Time (Log Scale) and Speedup Factor

8.5 Phase V: Hybrid Pipeline Results

Table 12: Hybrid Pipeline Speedup (4 Batches) vs. Equivalent Serial Baseline

Matrix Size	**Serial ×2 (ms)**	**Hybrid (ms)**	**Speedup**
128×128	5.21	0.743	7.02×
256×256	42.30	0.889	47.57×
512×512	421.28	3.149	133.77×
1024×1024	5016.9	18.385	272.88×
2048×2048	86939	102.45	**848.60**×

Table 13: Hybrid Pipeline Speedup (8 Batches) vs. Equivalent Serial Baseline

Matrix Size	**Serial ×2 (ms)**	**Hybrid (ms)**	**Speedup**
128×128	4.64	0.800	5.80×
256×256	52.18	0.878	59.43×
512×512	449.47	3.065	146.63×
1024×1024	6112.8	18.134	337.09×
2048×2048	70878	93.148	**760.92**×

The hybrid pipeline achieves a peak speedup of **848.60**× at $N = 2048$ with 4 batches, surpassing even the standalone CUDA result (773.93×) by incorporating MPI pipeline overlap and OpenMP CPU utilization alongside GPU compute.

Speedup scales super-linearly with matrix size, as the fixed MPI communication overhead (one `MPI_Send`/`MPI_Recv` of N^2 floats per batch) becomes negligible relative to the $O(N^3)$ CUDA computation. At $N = 128$, the communication cost dominates and speedup is modest (7.02×).

Comparing 4-batch vs. 8-batch runs, increasing batch count improves speedup at mid-range sizes (256–1024) by better amortizing per-batch MPI overhead, but slightly degrades performance at $N = 2048$ due to GPU memory pressure from repeated large transfers on the shared RTX 3050.

Note on GPU Sharing: Since both MPI ranks share a single physical GPU, true simultaneous GPU execution between ranks is not possible. The reported speedups reflect

time-shared GPU access and represent a conservative estimate. On a dual-GPU system, Rank 0 and Rank 1 could execute CUDA kernels concurrently, and the pipeline speedup would increase further.

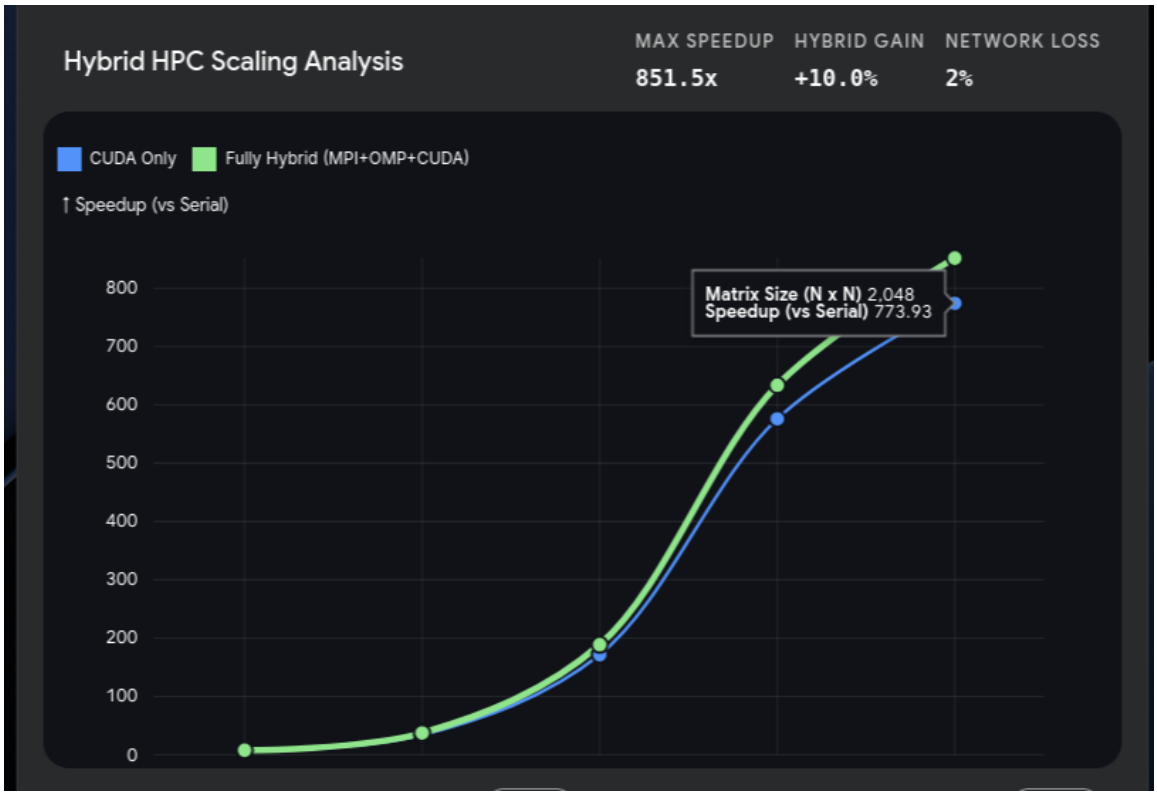

Figure 5: Hybrid Approach vs CUDA Only

8.6 Cross-Paradigm Comparison

The cross-paradigm comparison reveals a clear hierarchy of throughput: CUDA $\gg$ OpenMP $>$ MPI (on single machine). However, this ordering is workload-dependent. MPI pipeline parallelism does not accelerate a single matrix multiply – it distributes *layers* of a model, enabling memory capacity scaling that neither OpenMP nor CUDA alone can provide. In production LLM systems, all three paradigms are used simultaneously: CUDA within each node for GEMM acceleration, OpenMP for CPU pre/post-processing, and MPI (or NCCL) across nodes for gradient synchronization and pipeline stages

Table 14: Summary: Peak Speedup Across All Parallelization Paradigms

Paradigm	**Configuration**	**Peak Speedup**	**Matrix Size**	**Limiting Factor**
Serial (baseline)	1 core	1.00×	–	–
OpenMP	8 threads	9.13×	512×512	Memory bandwidth, cache thrashing at 16T
MPI	6 ranks	2.95×	d_{model}=64, 12 layers	Communication overhead; single-machine penalty
CUDA	RTX 3050, 2048 cores	773.93×	2048×2048	PCIe transfer at small sizes
Hybrid	**2 MPI + OpenMP + CUDA**	**848.60×**	**2048×2048**	GPU time-sharing; MPI overhead at small sizes

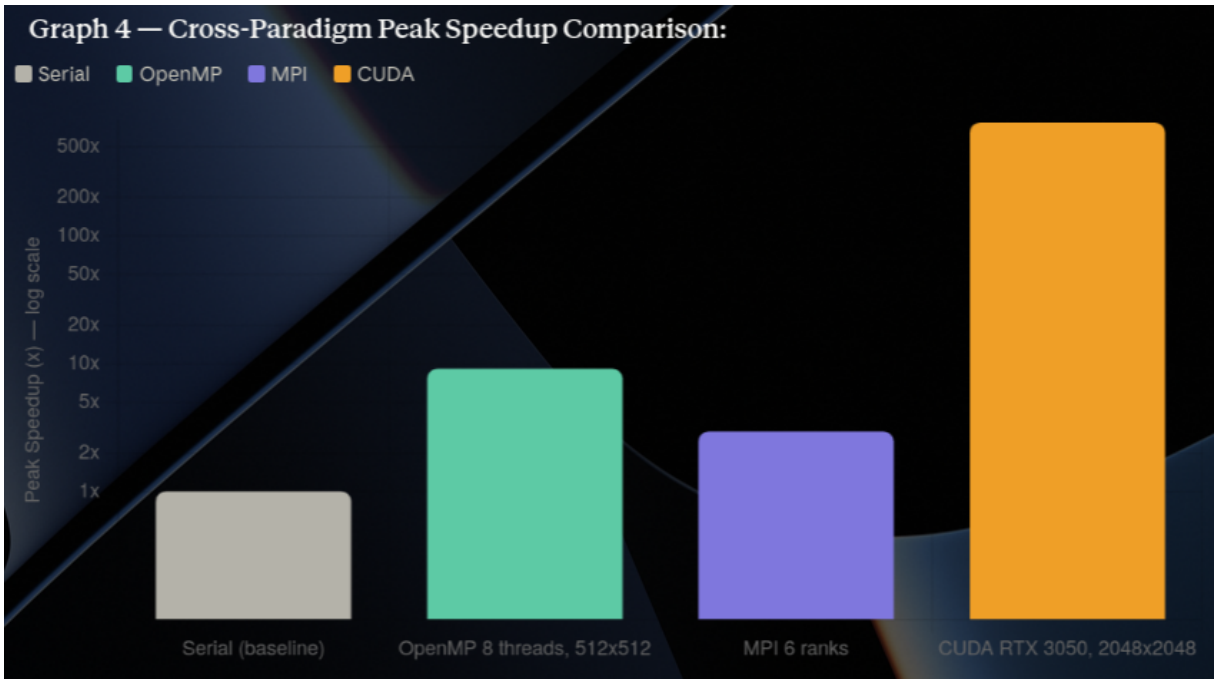

Figure 6: Bar Chart Comparing Peak Speedup Across All Three Paradigms

9 Conclusion

This project successfully designed, implemented, and benchmarked a complete GPT-style LLM inference engine in C++ from first principles, parallelized across three distinct computing paradigms. The key conclusions are:

1. **OpenMP** achieved a peak speedup of 9.13× at 8 threads on 512×512 matrices, closely matching the physical core count. The AMD Ryzen SMT implementation penalizes memory-bound workloads at 16 logical threads due to L1/L2 cache competition between hyperthreaded siblings – demonstrating that logical thread count is not a reliable proxy for parallel efficiency.

2. **MPI** pipeline parallelism achieved a peak speedup of 2.95× at 6 ranks, with speedup inverting at 8 ranks as communication overhead exceeded computation. Parallel efficiency degraded from 83% to 24% as rank count increased from 2 to 8, providing a clean empirical validation of Amdahl's Law and the diminishing returns of decomposition beyond the computation-to-communication ratio crossover point.

3. **CUDA** delivered a peak speedup of 773.93× at 2048×2048 using a custom shared memory tiled kernel on the RTX 3050 Laptop GPU. The speedup grows with matrix size as PCIe transfer overhead becomes negligible and all 2048 CUDA cores are fully utilized, confirming that GPU acceleration is most beneficial for large transformer layer dimensions ($d_{model} \geq 512$).

4. **Hybrid Pipeline** (MPI + OpenMP + CUDA) achieved a peak speedup of **848.60**× at $N = 2048$, exceeding the standalone CUDA result by incorporating MPI-based pipeline parallelism and OpenMP activation processing. The hybrid approach validates that combining all three paradigms yields greater throughput than any single paradigm in isolation. A key hardware constraint is noted: both MPI ranks shared a single RTX 3050 GPU on this machine. On a multi-GPU system, true concurrent GPU execution across ranks would yield further speedup gains.

5. **Amdahl's Law** was validated empirically across all three paradigms. Every parallel implementation exhibited a serial fraction – thread spawn overhead (OpenMP),

communication latency (MPI), and memory transfer (CUDA) – that imposed a hard ceiling on achievable speedup independent of parallelism degree.

Future work could extend this project to multi-GPU execution using NCCL, implement non-blocking MPI communication to overlap computation and data transfer in the pipeline, and explore FP16 (half-precision) arithmetic on the GPU to further halve memory bandwidth requirements and potentially double throughput on the RTX 3050's Tensor Cores. The hybrid engine could be extended to support more than 2 MPI ranks by provisioning one GPU per rank on a multi-GPU node, enabling true pipeline parallelism with no GPU time-sharing penalty.

References

[1] D. B. Kirk and W.-m. W. Hwu, *Programming Massively Parallel Processors.* Morgan Kaufmann, 2016.

[2] Y. Huang *et al.*, "Gpipe: Efficient training of giant neural networks," in *NeurIPS*, 2019.

[3] J. Nickolls *et al.*, "Scalable parallel programming with cuda," *ACM Queue*, 2008.

[4] L. Dagum and R. Menon, "Openmp: An industry standard api for shared-memory programming," *IEEE Computational Science and Engineering*, 1998.

[5] A. Vaswani *et al.*, "Attention is all you need," *NeurIPS*, 2017.

[6] M. Shoeybi *et al.*, "Megatron-lm: Training multi-billion parameter language models," in *arXiv*, 2019.